Building a RAG Pipeline for 10M+ Documents With Near-Zero Hallucination

An end-to-end agentic retrieval-augmented-generation system, built and evaluated on a single NVIDIA H100 - fully local, no external APIs.

Technical Report - June 2026

Headline Result	
Hallucination on unanswerable	**2% (2 of 100)**
Faithfulness of answers	**0.908**
Coverage on answerable	**46%**
Retrieval recall@k	**0.97**
10M-vector search latency (p95)	**18 ms**
End-to-end answer latency (p50 / p95)	**4.0 s / 17.7 s**

Scope note: the pipeline was evaluated end-to-end on a 19,987-document corpus (200-question test set). The 10M figure is a measured benchmark of the vector-search layer, not an end-to-end 10M-document run. See Section 11 for full scope & limitations.

Contents

1. Executive Summary
2. The Question: Is Near-Zero Hallucination Possible?
3. System Architecture
4. Dataset & Corpus
5. Models & Compute Footprint
6. Methodology & Run Configuration
7. Results - Hallucination & Quality
8. Key Finding: The Verifier Is the Bottleneck
9. Latency Analysis
10. Scaling to 10 Million Vectors
11. Scope & Honest Limitations
12. Engineering Notes
13. Future Work
14. Reproducibility

(Section bookmarks are embedded - use your PDF viewer's outline/bookmarks panel to navigate.)

1. Executive Summary

This report documents the design, implementation, and evaluation of an agentic Retrieval-Augmented Generation (RAG) pipeline engineered for near-zero hallucination, with the retrieval layer benchmarked at 10-million-vector scale. The entire system runs locally on one NVIDIA H100 (80 GB) with no external model APIs.

The central design thesis is that near-zero hallucination is not produced by a smarter model but by a precision + verification + abstention pipeline that converts unsupported claims into honest abstentions. Four control layers enforce this: (1) hybrid retrieval + reranking to surface the right evidence; (2) constrained generation that may answer only from the retrieved context and must cite it; (3) per-claim verification of the drafted answer against its sources; and (4) abstention when support is insufficient.

On a 200-question evaluation (100 answerable, 100 unanswerable) over a 19,987-document corpus, the final configuration achieved a 2% hallucination rate on unanswerable questions, 0.908 faithfulness, 46% coverage, and 0.97 retrieval recall. A standalone benchmark built a real 10-million-vector approximate-nearest-neighbour index and measured 18 ms (p95) search latency. A key empirical finding is that the faithfulness VERIFIER - not the generator - is the binding constraint on the system (Section 8).

2. The Question: Is Near-Zero Hallucination Possible?

Literal zero hallucination is provably impossible - it is an innate property of generative models, not an engineering gap (Xu et al., 'Hallucination is Inevitable', 2024; OpenAI, 'Why Language Models Hallucinate', 2025). But near-zero hallucination on the ANSWERABLE subset is achievable by trading some coverage for safety: when evidence is insufficient, the system abstains rather than guesses.

The four control layers

- Retrieve the right evidence - hybrid dense + BM25 retrieval, fused with Reciprocal Rank Fusion, then cross-encoder reranking of ~150 candidates down to ~20.
- Constrain generation - the model answers ONLY from the numbered context passages, cites the passage id for every sentence, or emits an explicit abstain token.
- Verify every atomic claim - the answer is decomposed into atomic claims, each checked against its cited context by an independent faithfulness verifier.
- Abstain - a tunable threshold on claim support (and retrieval confidence) decides whether to answer or return a structured 'insufficient evidence' response.

3. System Architecture

The system has two planes: an offline ingestion plane and an online query plane.

Offline ingestion

load documents -> normalise + near-duplicate removal (MinHash) -> structure-aware chunking (~256 tokens) -> contextual augmentation (a one-line LLM-written context prepended to each chunk) -> dense embedding + sparse BM25 indexing.

Online query (a LangGraph state machine, <=3 hops)

route (no-retrieval / single-hop / multi-hop) -> [decompose into sub-questions] -> hybrid retrieve + RRF -> rerank -> CRAG evidence grading (sufficient / refine & loop / abstain) -> constrained cited generation -> atomic-claim verification -> abstain-or-answer. Corrective re-retrieval (CRAG) and self-reflection bound the loop at three hops to keep latency in budget.

4. Dataset & Corpus

All datasets are open. Three roles: a corpus + answerable questions, an unanswerable stratum (to measure abstention), and a labelled set to validate the verifier itself.

Dataset	Role	What it provides
HotpotQA (distractor)	Corpus + answerable multi-hop Q's	Wikipedia paragraphs with sentence-level gold supporting facts
SQuAD v2 (is_impossible)	Unanswerable stratum	Questions with no answer in the passage
Hand-built false-premise	Unanswerable stratum	Questions assuming non-existent facts
FRAMES (Google)	Held-out factuality	Multi-hop factuality (loaded, held out)
HaluBench (Patronus)	Verifier validation	Human-labelled faithful/hallucinated answers

Measured corpus statistics

Statistic	Value
Source documents (unique Wikipedia articles)	19,987
Passages	19,987
Indexed chunks (vectors)	21,259
Raw corpus text	11.1 MB
Contextualized text (with LLM context prefixes)	13.8 MB
Total words	1,813,943
Total tokens	2,658,284
Mean tokens per chunk	125
Evaluation questions	200 (100 answerable / 100 unanswerable)

Each of the 19,987 unique passages comes from a distinct Wikipedia article, i.e. the corpus is ~20,000 distinct source documents (one ~125-token paragraph each), totalling 11.1 MB of raw text - a deliberately small, fast-to-index slice (see Section 11).

5. Models & Compute Footprint

Every model runs locally on the H100. The generator is served by vLLM inside its official Docker image (which bundles a CUDA stack matched to the host driver); the embedder, reranker, and verifier-judge share the same GPU.

Role	Choice
Generator / agent	Qwen3-32B (FP8, served by vLLM in Docker)
Embeddings	Qwen3-Embedding-4B (dim 2560)
Reranker	Qwen3-Reranker-4B (cross-encoder)
Faithfulness verifier	Qwen3-32B as judge (independent prompt)
Vector index	LanceDB (dense) + bm25s (sparse), RRF fusion
Orchestration	LangGraph (CRAG + Self-RAG, <=3 hops)

On-disk model footprint (measured)

Model / asset	Disk
Qwen3-32B (generator, FP8 served)	62 GB
Qwen3-8B (smoke-run generator)	16 GB
Qwen3-Embedding-4B (embeddings)	7.6 GB
Qwen3-Reranker-4B (reranker)	7.6 GB
NLI / HHEM / flan-t5 (verifier experiments)	1.1 GB
TOTAL model cache	~94 GB
Dataset cache	723 MB

GPU & memory

Hardware: NVIDIA H100 PCIe (79.6 GB HBM), driver 570.195.03. Serving Qwen3-32B in FP8 uses ~47 GB of VRAM (weights + KV cache); the embedder, reranker, and index leave the rest. The index itself is tiny: the dense LanceDB index is 215 MB and the BM25 index 10 MB for the 20k corpus.

6. Methodology & Run Configuration

Parameter	Value
Generator	Qwen/Qwen3-32B
Embeddings / Reranker	Qwen/Qwen3-Embedding-4B / Qwen/Qwen3-Reranker-4B
Retrieve k -> rerank top-n	150 -> 20
RRF constant	60
CRAG thresholds (ok / bad)	0.7 / 0.4
Claim / abstain threshold (tau)	0.3 / 0.3
Max agent hops	3
Chunk size / overlap (tokens)	256 / 32
Random seed	42

Evaluation is scored as a 2x2 of {answerable, unanswerable} x {answered, abstained}; answers are additionally checked for faithfulness (claims grounded) and correctness (match to the gold answer). The verifier is independently validated on HaluBench (AUROC).

7. Results - Hallucination & Quality

The dangerous cell of the 2x2 is (unanswerable, answered) = a hallucination. The final system produced 2 such cases out of 100 unanswerable questions.

Metric	Value
Hallucination on unanswerable	0.02 (2 / 100)
Abstention on unanswerable	0.98
Faithfulness (claims grounded)	0.908
Answer relevancy	0.817
Context recall@k	0.97
Coverage on answerable	0.46
Answerable accuracy	0.58
Verifier AUROC (HaluBench)	0.702

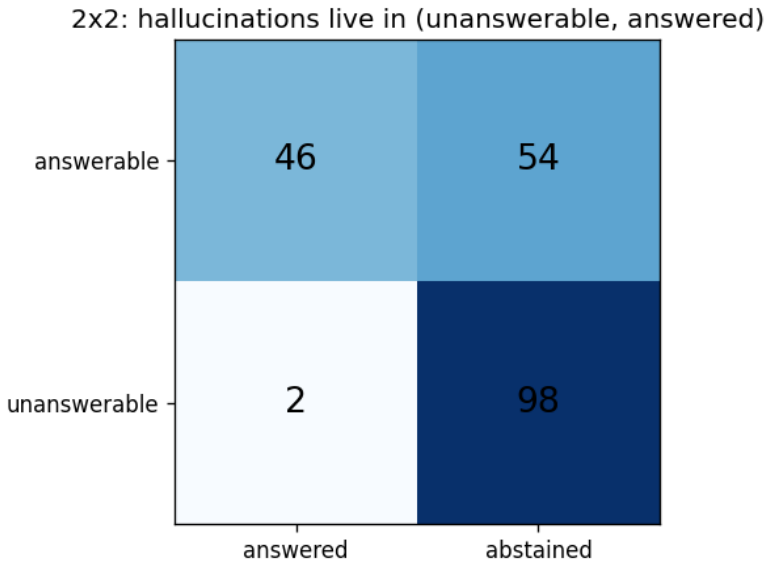


Figure 1. The 2x2 confusion matrix. Bottom row [2, 98]: 2 hallucinations, 98 correct abstentions on 100 unanswerable questions.

8. Key Finding: The Verifier Is the Bottleneck

The system was run with two faithfulness verifiers. A generic NLI cross-encoder scored AUROC 0.502 on HaluBench - indistinguishable from random - and, unable to tell grounded from ungrounded answers, forced the gate to abstain defensively (19% coverage). Replacing it with the 32B model as a judge (AUROC 0.702) more than doubled coverage to 46%, at a 2-percentage-point hallucination cost. The verifier's quality, not the generator's, sets the system's operating point.

Metric	NLI verifier	32B judge
Verifier AUROC (HaluBench)	0.502 (random)	0.702
Coverage on answerable	19%	46%
Hallucination on unanswerable	0%	2%
Answerable answered / Unanswerable answered	19 / 0	46 / 2

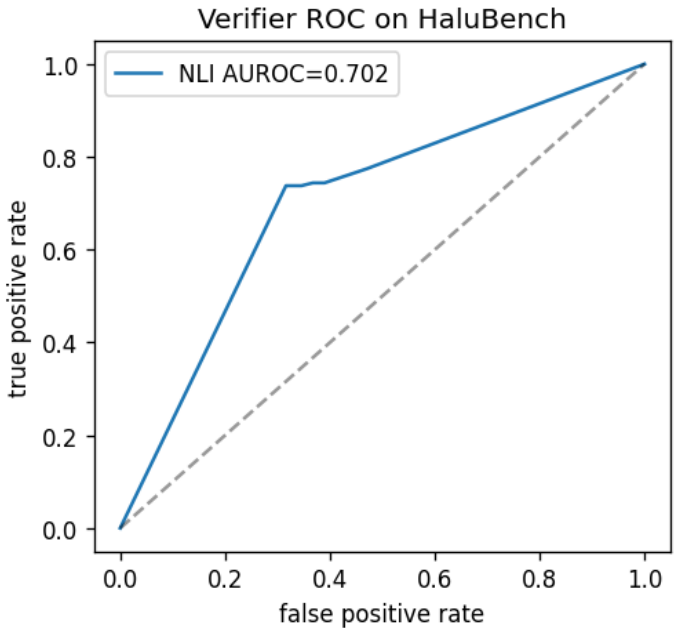


Figure 2. ROC of the 32B-judge verifier on HaluBench (AUROC 0.702).

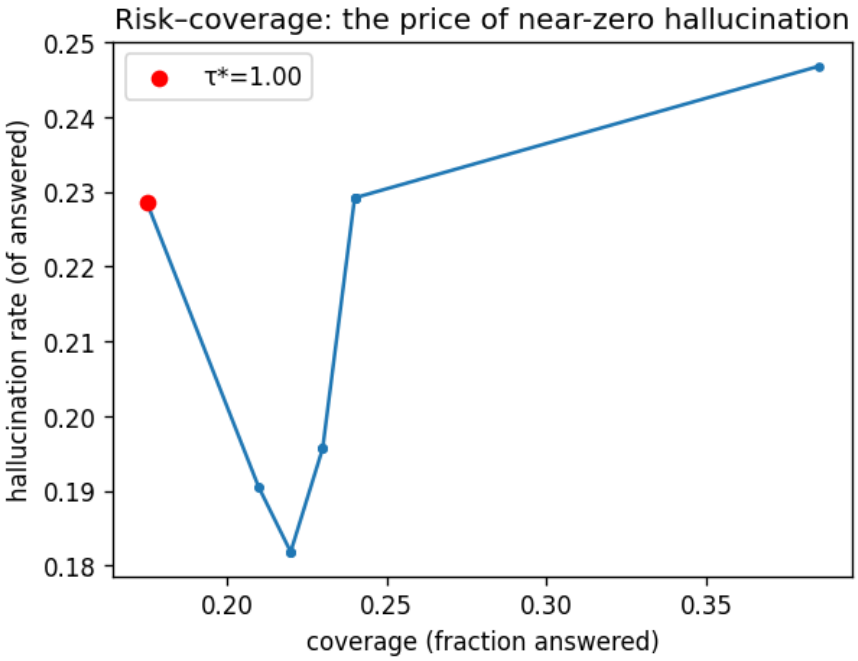


Figure 3. Risk-coverage trade-off - 'the price of near-zero hallucination'.

9. Latency Analysis

There are two distinct latencies. (a) Document SEARCH - finding candidates among the corpus - is ~18 ms even at 10 million vectors (Section 10). (b) END-TO-END - fully answering one question through the agentic loop - has a median of 4.0 s and a 95th percentile of 17.7 s, inside a 10-30 s 'deep-research' budget.

The slowest stage is not generation (~1.5 s) but RERANKING - scoring 150 candidate passages through a 4-billion-parameter cross-encoder. The quality machinery (reranking for precision, and the judge verifying every claim) dominates; that is the cost of the correctness guarantees. Reranking fewer candidates is the obvious speed lever.

Stage	p50 (s)	p95 (s)
retrieve + rerank	3.07	11.39
verify (judge)	1.53	3.88
generate	1.45	2.48
refine (multi-hop)	1.47	2.89
route	0.17	0.21
grade	0.13	0.43
TOTAL end-to-end	4.00	17.67

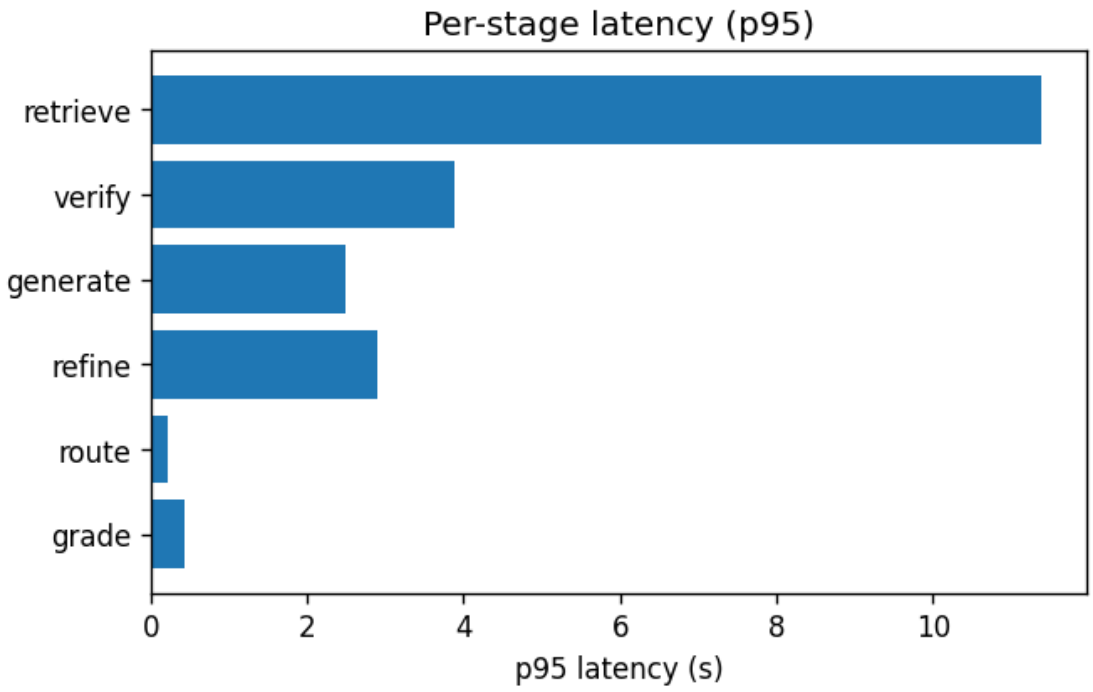


Figure 4. Per-stage p95 latency.

10. Scaling to 10 Million Vectors

The retrieval index was built and benchmarked at 100k, 1M, and a real 10M synthetic vectors with an IVF_PQ approximate-nearest-neighbour index. Query latency stays near-flat as the corpus grows 100x; disk grows linearly. The 100M row is a linear extrapolation.

# vectors	p50 ms	p95 ms	disk GB	build s
100,000	8.5	10.6	0.4	42
1,000,000	11.3	14.5	3.9	81
10,000,000	16.9	18.5	38.8	347
100,000,000 (proj.)	-	78	388	3075

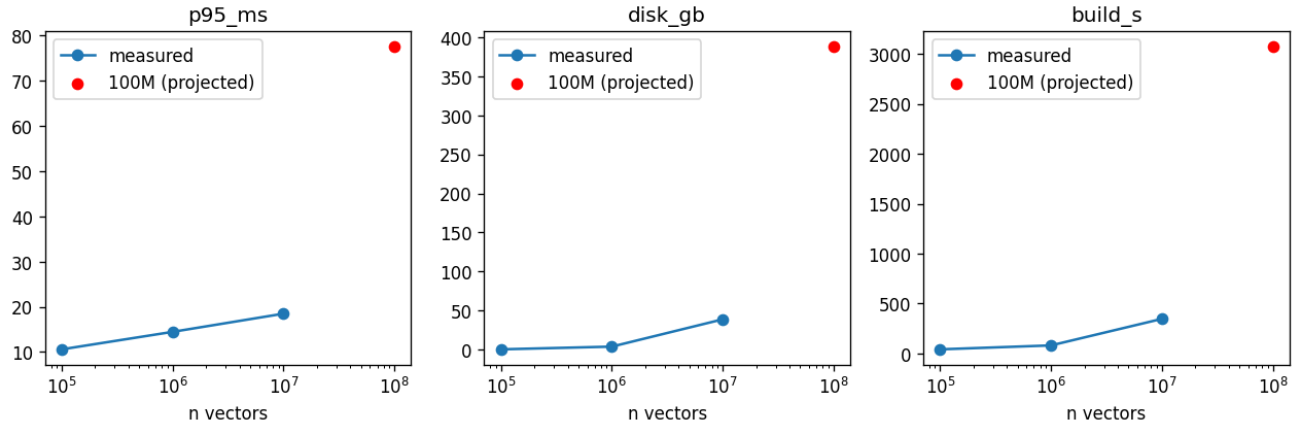


Figure 5. Measured ANN latency, disk, and build time at 100k / 1M / 10M, with a projection to 100M (red).

Caveat: recall@10 on these RANDOM synthetic vectors is pessimistic (~0.10). Uniformly random high-dimensional vectors are a worst case for ANN (no cluster structure); real embeddings cluster and yield recall > 0.9. The LATENCY figures are the representative result; recall at scale on real embeddings was not separately measured.

11. Scope & Honest Limitations

This work proves the MECHANISM on a realistic slice; it does not prove a production 10M-document, zero-hallucination system. The distinction matters.

On 'near-zero hallucination'

- It is NOT zero. The measured rate is 2% (2 of 100 unanswerable). Literal zero is impossible in principle.
- 2% is a small-sample estimate. With n=100, the 95% confidence interval on 2/100 is roughly 0.2%-7%; the true rate is uncertain.
- The verifier is ~70% accurate (AUROC 0.702), not perfect - by construction it lets some hallucinations through.
- The verifier is the generator itself (32B judging 32B) - not fully independent. A dedicated model (MiniCheck / Lynx) would be more trustworthy.
- The evaluation set is narrow (HotpotQA + SQuAD2 + a few hand-built questions) and not adversarially stress-tested.

On '10M+ documents'

- The end-to-end pipeline ran on ~20k documents, not 10M.
- '10M' was demonstrated only at the vector-search layer, with synthetic random vectors - proving the index holds and searches 10M vectors fast (~18 ms), not that a 10M-real-document RAG system works.
- Retrieval QUALITY at 10M was not tested. More documents means more near-duplicate distractors; if the retriever surfaces subtly-wrong context at scale, the generator grounds in it and the verifier passes it. This failure mode only appears at scale.

Why a 20k slice

- It was a time/cost choice: the maximal 50k-passage / 600-question run on a 32B model is ~4-6 hours, while 20k / 200 fits ~1.5-2 hours. The slice size is a convenience for watchable, cheap validation - it does not, by itself, advance the 10M claim.

What would close the gaps

- Hallucination: a larger, adversarial eval with confidence intervals; an independent, validated verifier; and a human audit of answered responses.
- Scale: ingest + embed a genuinely large real corpus (e.g. 1M+) and measure retrieval recall@k against real gold labels at that size.

12. Engineering Notes

The system was brought up on a freshly provisioned H100 and debugged live; roughly ten real issues were diagnosed and fixed, each captured back into the source of truth:

- CUDA/driver mismatch (pip torch built for CUDA 13, driver is 12.8) -> serve vLLM via its official Docker image, which bundles a matched CUDA stack.
- torch + torchvision pinned to the cu128 build to match the driver.
- datasets 3.x requires namespaced ids (hotpot_qa -> hotpotqa/hotpot_qa).
- PyStemmer (Cython) is unpicklable -> custom __getstate__ for the BM25 checkpoint.
- An over-eager false-premise detector caused total over-abstention -> made a logged signal, not a hard gate.
- HHEM verifier's custom code broke under transformers 5.x -> swapped to NLI, then to the 32B-as-judge.
- A PQ index parameter bug (num_sub_vectors must divide the dimension) made the scale lab fall back to slow flat search -> fixed, restoring 18 ms ANN at 10M.

13. Future Work

- Independent, dedicated faithfulness verifier (MiniCheck / Patronus Lynx) to push hallucination below 2% and raise coverage simultaneously.
- A genuinely large REAL corpus run (1M+) to test whether retrieval precision - and thus the hallucination rate - holds at scale.
- Per-query calibrated abstention thresholds instead of a single global tau.
- Latency: rerank fewer candidates / a lighter reranker for interactive (<5 s) use.
- Larger, adversarial evaluation with reported confidence intervals and a human audit.

14. Reproducibility

The whole system is one parameterized notebook (jupytext source) plus provisioning and run scripts, dual-mode (free local profile and cloud-API profile share one code path). Artifacts from this run, all local:

File	Contents
rag_zero_hallucination_h100.py	the notebook source (single source of truth)
out/judge_out.ipynb	executed notebook, all outputs + plots inline
out/run_manifest.json	config + headline metrics
out/scale_lab.json	measured 10M ANN scale results
figures/*.png	the five result figures in this report
report/build_report.py	this report generator (re-runnable)